



UNINASSAU

# Algoritmos

## Estruturas de Repetição e Listas em Python

Profº: Joseph Donald



- Repetições representam a base de vários programas.
- Uma das estruturas de repetição do Python é o **while**, que repete um bloco quando a condição for verdadeira.
- Veja como fica a sintaxe:

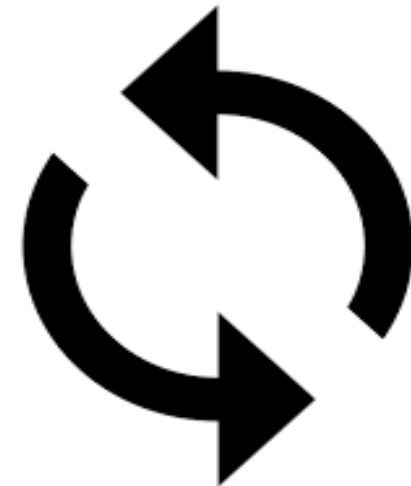
**while** *<condição>*:  
    *<bloco de execução>*



- Vamos fazer um exemplo para exibir 3 números utilizando o while.

```
>>> x = 1
>>> while x <= 3:
    print(x)
    x = x + 1
```

```
1
2
3
>>>
```





## Outro exemplo:

- Elabore um programa que exiba a tabuada de multiplicar de um número inserido pelo usuário:

```
num = int(input("Tabuada de: "))
```

```
x = 0
```

```
while x <= 10:
```

```
    print(x * num)
```

```
    x = x + 1
```



## Exemplo prático

- Crie um programa em Python que faça a contagem regressiva do início da largada de fórmula 1. Peça para o usuário inserir o valor inicial.

```
import time

num = int(input("Informe o número inicial da contagem:"))

while num >= 0:

    if num == 0:
        print("Go Go Go!!!")
    else:
        time.sleep(1)
        print(num)

    num = num - 1
```



## Repetições - for

- A instrução **for** em Python funciona de forma semelhante ao comando **while**, porém o for contém seu próprio contador.
- Normalmente é utilizada para percorrer um vetor (ou lista)
- Utiliza a cláusula **range** para customizar o início, fim e também o contador do loop.

Exemplo:

```
for num in range(5):  
    print(num)
```



# Repetições - for

- Observe que o ***range*** recebe um valor que usará como limite para esse loop.
- Ele também permite que seja inserido o valor inicial do loop e os incrementos desse loop.
- Obs: o ***for*** não inclui o valor limite na contagem.

Exemplo: 

```
for num in range(5):  
    print(num)
```

***início***      ***incremento***

↓              ↓

```
for num in range(1, 10, 1):  
    print(num)
```

                  ↑

***limite***

```
for num in range(9, -1, -1):  
    print(num)
```



**Exemplo:** Utilizando o comando **for** faça um programa que verifique e exiba quais números são **pares** no intervalo entre 1 e 20.

Dica: São apenas 3 linhas de código.

```
for num in range(1, 21):  
    if num % 2 == 0:  
        print(num)
```





## Desafio 1!

**Exemplo:** Faça um programa que leia 5 números e informe o maior número.



## Desafio 2!

**Exemplo:** Faça um programa que leia 5 números e informe a soma e a média dos números.



# Listas

- Lista é um tipo de variável que permitem o armazenamento de vários valores acessados por um índice, mas com algumas peculiaridades.
- Uma lista pode estar vazia, conter 0 (zero) elementos ou mais elementos de um tipo ou de tipos diversos.
- O tamanho da lista é variável e correspondente a quantidade de elementos que ela contém.



## Sintaxe para preenchimento:

- Exemplo de lista vazia:  
*L = []*
- Lista com 3 elementos:  
*frutas = ['Maça', 'Abacate', 'Tangerina']*
- Lista com elementos de diferentes tipos:  
*itens = ['Fusca', 8.5, 100]*

## Sintaxe para impressão:

- Exemplo de lista vazia:  
*print(L)*
- Lista com 3 elementos:  
*print(frutas)*  
*print(frutas[1])*
- Lista com elementos de diferentes tipos:  
*print(itens)*  
*print(itens[2])*

## Listas

```
>>> L = []
>>> print(L)
[]
>>> frutas = ['Maça', 'Abacate', 'Tangerina']
>>> print(frutas)
['Maça', 'Abacate', 'Tangerina']
>>> print(frutas[1])
Abacate
>>> itens = ['Fusca', 8.5, 100]
>>> print(itens)
['Fusca', 8.5, 100]
>>> print(itens[2])
100
>>> |
```



# Listas

## Exemplo prático

Utilizando o Python crie as 2 listas abaixo e depois imprima o resultado:

- Produtos de limpeza

- ✓ Sabonete
- ✓ Shampoo
- ✓ Desinfetante.

- Alimentos

- ✓ Arroz
- ✓ Feijão
- ✓ Carne

```
>>> prodLimpeza = ['Sabonete', 'Shampoo', 'Desinfetante']
>>> print(prodLimpeza)
['Sabonete', 'Shampoo', 'Desinfetante']
>>> alimentos = ['Arroz', 'Feijão', 'Carne']
>>> print(alimentos)
['Arroz', 'Feijão', 'Carne']
>>> listaSupermercado = [prodLimpeza, alimentos]
>>> print(listaSupermercado)
[['Sabonete', 'Shampoo', 'Desinfetante'], ['Arroz', 'Feijão', 'Carne']]
>>> |
```



# Listas

Adicionando elementos numa lista já existente:

- Para inserir um item no final da lista, podemos utilizar a função ***append( )***.

Ex:

```
alimentos.append('Macaxeira')
```

- Para inserirmos um item numa determinada posição da lista, podemos utilizar a função ***insert( )***. Ex:

```
alimentos.insert(0, 'Macarrão')
```

```
>>> alimentos = ['Arroz', 'Feijão', 'Carne']
>>> alimentos.append('Macaxeira')
>>> print(alimentos)
['Arroz', 'Feijão', 'Carne', 'Macaxeira']
>>> alimentos.insert(0, 'Macarrão')
>>> print(alimentos)
['Macarrão', 'Arroz', 'Feijão', 'Carne', 'Macaxeira']
>>> |
```



# Listas

Alterando elementos de uma lista:

- Para alterar um elemento basta substituí-lo através do seu índice. Ex:

*alimentos[4] = 'Frango'*

```
['Macarrão', 'Arroz', 'Feijão', 'Carne', 'Macaxeira']
```

```
>>> alimentos[4] = 'Frango'
```

```
>>> print(alimentos)
```

```
['Macarrão', 'Arroz', 'Feijão', 'Carne', 'Frango']
```

```
>>> |
```



# Listas

Excluindo elementos de uma lista:

- Para remover um item da lista, utilizamos a função ***del()***
- Nela simplesmente informamos a lista e o índice do elemento que queremos deletar. Ex:

*del(alimentos[3])*

```
['Macarrão', 'Arroz', 'Feijão', 'Carne', 'Frango']  
>>> del(alimentos[3])  
>>> print(alimentos)  
['Macarrão', 'Arroz', 'Feijão', 'Frango']  
>>>
```





# Listas

Para remover todos os elementos de uma lista:

- Se precisamos limpar ou 'zerar' uma lista, então basta utilizar a função ***clear()***
- Então ela eliminará todos os itens da lista deixando a mesma vazia novamente.

Ex: *alimentos.clear()*

```
>>> print(alimentos)
['Macarrão', 'Arroz', 'Feijão', 'Frango']
>>> alimentos.clear()
>>> print(alimentos)
[]
>>> |
```



## Desafio 1

- Crie uma lista chamada “alunos”
- Utilizando estruturas de repetição e a função ***append*** insira nessa lista o nome dos alunos da sala, incluindo o seu.
- Agora remova o seu nome da lista com o comando ***del***
- Insira seu nome como primeiro da lista usando o comando ***insert***



# Listas - Ordenação

- Para ordenar os itens de uma lista, podemos utilizar a função ***sort()***

Ex: *alunos.sort()*

- E para colocarmos na ordem inversa utilizamos a função ***reverse()***

Ex: *alunos.reverse()*

```
>>> alunos = ['Huguinho', 'Zezinho', 'Luizinho', 'Monica', 'Cebolinha']
>>> alunos
['Huguinho', 'Zezinho', 'Luizinho', 'Monica', 'Cebolinha']
>>> alunos.sort()
>>> alunos
['Cebolinha', 'Huguinho', 'Luizinho', 'Monica', 'Zezinho']
>>> alunos.reverse()
>>> alunos
['Zezinho', 'Monica', 'Luizinho', 'Huguinho', 'Cebolinha']
>>> |
```



- Há também outras funções bem interessantes com as listas, como:

```
>>> numeros = [1, 2 , 3, 4, 5, 6]
>>> sum(numeros)
21
>>> len(numeros)
6
>>> max(numeros)
6
>>> min(numeros)
1
```

***sum*** : Soma os elementos numéricos da lista

***len***: Retorna o tamanho da lista

***max***: Retorna o maior elemento da lista

***min***: Retorna o menor elemento da lista



- Para escrever ou ler os itens de uma lista, podemos utilizar-se também das estruturas de repetição. Ex:

```
carros = []
quantidade = int(input('Quantos carros deseja cadastrar?'))
cont = 0
while cont < quantidade:
    carro = input(f'Digite o nome do {cont+1}º carro:')
    carros.append(carro)
    cont += 1

print(f'Lista completa:{carros}')

for i, carro in enumerate(carros):
    print(f'{i+1}º Carro:{carro}')
```

|



## Desafio 2

- Refaça o algoritmo do exemplo anterior utilizando a estrutura de repetição **FOR**.